

intfloat/multilingual-e5-base   like 313

 Sentence Similarity  sentence-transformers  PyTorch  ONNX

 Safetensors  OpenVINO  94 languages xlm-roberta mteb

Sentence Transformers  Eval Results  text-embeddings-inference  arxiv:4 papers

 License: mit

 Train  Deploy  Use this model

 Model card  Files  xet  Community 30

Downloads last month
1,417,585



 Safetensors 

Model size 278M params Tensor type I64 · F32  Files info

 Inference Providers 

 HF Inference API

 Sentence Similarity

Source Sentence

Your sentence here...

Sentences to compare to

Your sentence here...

Your sentence here...

Add Sentence

Generate

 View Code

 Maximize

 Model tree for intfloat/multilingual-e5-base

Finetunes 92 models

Quantizations 15 models

Collection including `intfloat/multilingual-e5-base`

Multilingual E5 Text Embeddings

 Collection

5 items • Updated Feb 17 •  7

Evaluation results

accuracy on MTEB AmazonCounterfactualClassification (en)	test set	self-reported	78.970
ap on MTEB AmazonCounterfactualClassification (en)	test set	self-reported	43.694
f1 on MTEB AmazonCounterfactualClassification (en)	test set	self-reported	73.381
accuracy on MTEB AmazonCounterfactualClassification (de)	test set	self-reported	71.724
ap on MTEB AmazonCounterfactualClassification (de)	test set	self-reported	82.221
f1 on MTEB AmazonCounterfactualClassification (de)	test set	self-reported	69.955
accuracy on MTEB AmazonCounterfactualClassification (en-ext)	test set	self-reported	79.655
ap on MTEB AmazonCounterfactualClassification (en-ext)	test set	self-reported	28.508
f1 on MTEB AmazonCounterfactualClassification (en-ext)	test set	self-reported	66.845
accuracy on MTEB AmazonCounterfactualClassification (ja)	test set	self-reported	73.330

Expand 1855 evaluations

 View on Papers With Code

Multilingual-E5-base

Multilingual E5 Text Embeddings: A Technical Report. Liang Wang, Nan Yang, Xiaolong Huang, Linjun Yang, Rangan Majumder, Furu Wei, arXiv 2024

This model has 12 layers and the embedding size is 768.

Usage

Below is an example to encode queries and passages from the MS-MARCO passage ranking dataset.

```
import torch.nn.functional as F
```

```

from torch import Tensor
from transformers import AutoTokenizer, AutoModel

def average_pool(last_hidden_states: Tensor,
                  attention_mask: Tensor) -> Tensor:
    last_hidden = last_hidden_states.masked_fill(~attention_mask[..., None].bool(), 0.0)
    return last_hidden.sum(dim=1) / attention_mask.sum(dim=1)[..., None]

# Each input text should start with "query: " or "passage: ", even for non-English
# For tasks other than retrieval, you can simply use the "query: " prefix.
input_texts = ['query: how much protein should a female eat',
               'query: 南瓜的家常做法',
               "passage: As a general guideline, the CDC's average requirement of",
               "passage: 1.清炒南瓜丝 原料:嫩南瓜半个 调料:葱、盐、白糖、鸡精 做法: 1、南瓜"]

tokenizer = AutoTokenizer.from_pretrained('intfloat/multilingual-e5-base')
model = AutoModel.from_pretrained('intfloat/multilingual-e5-base')

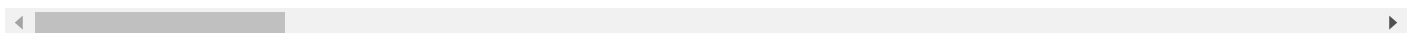
# Tokenize the input texts
batch_dict = tokenizer(input_texts, max_length=512, padding=True, truncation=True, return_tensors='pt')

outputs = model(**batch_dict)
embeddings = average_pool(outputs.last_hidden_state, batch_dict['attention_mask'])

# normalize embeddings
embeddings = F.normalize(embeddings, p=2, dim=1)
scores = (embeddings[:2] @ embeddings[2:].T) * 100
print(scores.tolist())

```

🔗 Supported Languages



This model is initialized from [xlm-roberta-base](#) and continually trained on a mixture of multilingual datasets. It supports 100 languages from xlm-roberta, but low-resource languages may see performance degradation.

[🔗 Training Details](#)

Initialization: [xlm-roberta-base](#)

First stage: contrastive pre-training with weak supervision

Dataset	Weak supervision	# of text pairs
Filtered mC4	(title, page content)	1B
CC News	(title, news content)	400M
NLLB	translation pairs	2.4B
Wikipedia	(hierarchical section title, passage)	150M
Filtered Reddit	(comment, response)	800M
S2ORC	(title, abstract) and citation pairs	100M
Stackexchange	(question, answer)	50M
xP3	(input prompt, response)	80M
Miscellaneous unsupervised SBERT data	-	10M

Second stage: supervised fine-tuning

Dataset	Language	# of text pairs
MS MARCO	English	500k
NQ	English	70k
Trivia QA	English	60k
NLI from SimCSE	English	<300k
ELI5	English	500k

Dataset	Language	# of text pairs
<u>DuReader Retrieval</u>	Chinese	86k
<u>KILT Fever</u>	English	70k
<u>KILT HotpotQA</u>	English	70k
<u>SQuAD</u>	English	87k
<u>Quora</u>	English	150k
<u>Mr. TyDi</u>	11 languages	50k
<u>MIRACL</u>	16 languages	40k

For all labeled datasets, we only use its training set for fine-tuning.

For other training details, please refer to our paper at <https://arxiv.org/pdf/2402.05672>.

🔗 Benchmark Results on Mr. TyDi

Model	Avg MRR@10		ar	bn	en	fi	id	ja	ko	ru	sw	te	th
BM25	33.3		36.7	41.3	15.1	28.8	38.2	21.7	28.1	32.9	39.6	42.4	41.7
mDPR	16.7		26.0	25.8	16.2	11.3	14.6	18.1	21.9	18.5	7.3	10.6	13.5
BM25 + mDPR	41.7		49.1	53.5	28.4	36.5	45.5	35.5	36.2	42.7	40.5	42.0	49.2
multilingual-e5-small	64.4		71.5	66.3	54.5	57.7	63.2	55.4	54.3	60.8	65.4	89.1	70.1
multilingual-e5-base	65.9		72.3	65.0	58.5	60.8	64.9	56.6	55.8	62.7	69.0	86.6	72.7
multilingual-e5-large	70.5		77.5	73.2	60.8	66.8	68.5	62.5	61.6	65.8	72.7	90.2	76.2

🔗 MTEB Benchmark Evaluation

Check out [unilm/e5](#) to reproduce evaluation results on the [BEIR](#) and [MTEB benchmark](#).

🔗 Support for Sentence Transformers

Below is an example for usage with sentence_transformers.

```
from sentence_transformers import SentenceTransformer
model = SentenceTransformer('intfloat/multilingual-e5-base')
input_texts = [
    'query: how much protein should a female eat',
    'query: 南瓜的家常做法',
    "passage: As a general guideline, the CDC's average requirement of protein for",
    "passage: 1.清炒南瓜丝 原料:嫩南瓜半个 调料:葱、盐、白糖、鸡精 做法: 1、南瓜用刀薄薄的削开",
]
embeddings = model.encode(input_texts, normalize_embeddings=True)
```

Package requirements

```
pip install sentence_transformers~=2.2.2
```

Contributors: [michaelfeil](#)

🔗 FAQ

1. Do I need to add the prefix "query: " and "passage: " to input texts?

Yes, this is how the model is trained, otherwise you will see a performance degradation.

Here are some rules of thumb:

- Use "query: " and "passage: " correspondingly for asymmetric tasks such as passage retrieval in open QA, ad-hoc information retrieval.

- Use "query: " prefix for symmetric tasks such as semantic similarity, bitext mining, paraphrase retrieval.
- Use "query: " prefix if you want to use embeddings as features, such as linear probing classification, clustering.

2. Why are my reproduced results slightly different from reported in the model card?

Different versions of `transformers` and `pytorch` could cause negligible but non-zero performance differences.

3. Why does the cosine similarity scores distribute around 0.7 to 1.0?

This is a known and expected behavior as we use a low temperature 0.01 for InfoNCE contrastive loss.

For text embedding tasks like text retrieval or semantic similarity, what matters is the relative order of the scores instead of the absolute values, so this should not be an issue.

Citation

If you find our paper or models helpful, please consider cite as follows:

```
@article{wang2024multilingual,  
  title={Multilingual E5 Text Embeddings: A Technical Report},  
  author={Wang, Liang and Yang, Nan and Huang, Xiaolong and Yang, Linjun and Majumdar, Anirban},  
  journal={arXiv preprint arXiv:2402.05672},  
  year={2024}  
}
```

Limitations

Long texts will be truncated to at most 512 tokens.